Teaching a Robot Arm to Pick and Place a Cube

Pick and Place in MuJoCo with Inverse Kinematics – ITR (ME 639) Challenge 3

Prachi Jindal · Mechanical Engineering, IIT Gandhinagar

Code + README: github.com/prachiiii27/ITR_Prachi/…/Challenge_3 · 3-min video: youtu.be/9ps0CDkTn_A

1. Introduction and Goals

In this challenge I made a robot arm pick up a small cube from a table and drop it into a tray, inside the MuJoCo simulator. The robot is a **Franka Emika Panda**: an arm with 7 rotating joints and a two-finger gripper. Every time, the cube starts at a new random place and angle on the table.

The hard part is a question called **inverse kinematics (IK)**: I know where the gripper needs to be, but what angles should the 7 joints take to get it there? I solved this in four ways and compared them on exactly the same cubes:

- **Mink** – a ready-made IK library (*github.com/kevinzakka/mink*), used as the baseline.
- **Pseudo-inverse** – the simplest textbook method (my method of choice).
- **Damped Least Squares (DLS)** – a safer version that stays calm near difficult arm poses.
- **Quadratic Programming (QP)** – the same idea, but with hard rules that joints may not cross their limits or move too fast.

My goals were: find out where the arm can reach and design the table from that; run many random trials and record everything; avoid the cube hitting the tray; compare the four IK methods fairly; and study what goes wrong and how to fix it.

2. Methods

2.1 The simulated world

I built the scene in Python (MuJoCo's *MjSpec*) so all sizes come from one settings file. It has the Panda, a table, a 16 × 16 cm tray with 5 cm walls, and a 4 cm, 50 g cube. The cube is a free body, so the gripper really has to hold it by friction – it can slip or be knocked off, just like in real life. I added a point between the two fingers, called the **TCP** (tool centre point); every IK target is a position and angle for this point. I also turned on gravity compensation, because without it the arm sagged a few millimetres and almost touched the table.

2.2 Where can the arm reach? (workspace analysis)

Before placing a table I had to know where the gripper can actually work. I took a grid of points every 5 cm and, at each point, asked the IK to reach it with the gripper pointing straight down, turned to 8 different angles (every 45°). A point counted only if the IK found an answer inside the joint limits and the elbow and wrist stayed at least 5 cm above the table. This gives two maps:

- **Task workspace** – points the gripper can reach at *at least one* angle (light blue in Fig. 1).
- **Dexterous workspace** – points it can reach at *every* angle (dark blue). Here a cube can be picked up whichever way it is turned, so this is where the cube should be.

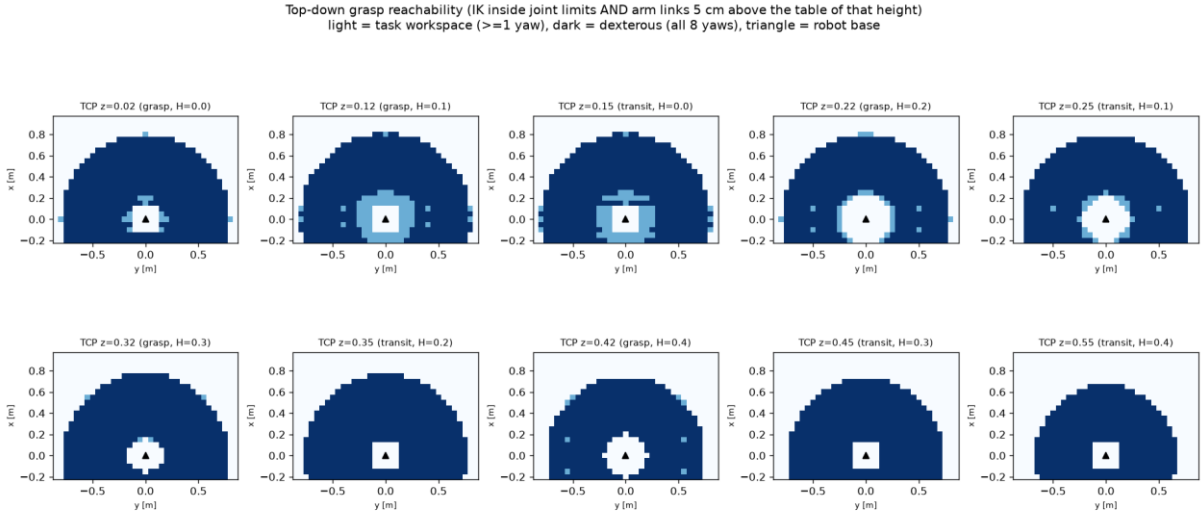


Fig. 1 – Reachability at the grasp height and travel height for each table height I tried (light = task workspace, dark = dexterous workspace, triangle = robot base).

2.3 Designing the table

I tried five table heights (0 to 0.4 m). For each, I looked for the biggest rectangle that is dark blue at both the grasp height and the travel height above it, and that keeps 30 cm away from the robot base. The table below shows my choice and why.

Parameter	Symbol	Value	Why
Table length	L	0.35 m (x 0.30 → 0.65)	Longest stretch that stays inside the dexterous area, starting clear of the robot base
Table width	W	0.90 m (y −0.45 → 0.45)	Widest stretch that stays dexterous – any cube angle can be picked
Table height	H	0.10 m	Gives the largest usable area (0.38 m²). The floor (H = 0) tied, but a real table keeps the arm's links further from it. Higher tables lose area: 0.34 m² at 0.2 m, 0.23 m² at 0.4 m

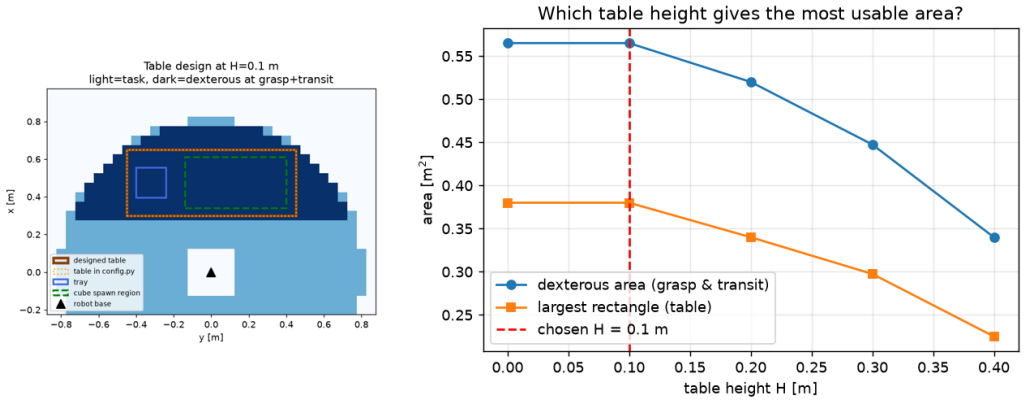


Fig. 2 – Left: final table (brown), tray (blue) and cube spawn area (green) on the dexterous map. Right: usable area for each table height.

2.4 One episode, step by step

Each trial (an **episode**) goes like this:

- **Random cube:** x between 0.34 and 0.61 m, y between −0.14 and 0.40 m, and any angle from −180° to 180°. Episode number i always uses random seed i, so all four IK methods get exactly the same 25 cubes.
- **Plan:** the gripper path is split into six straight moves – above the cube (12 cm up) → down to the cube → close → lift → over the tray → down into the tray → open → move away. Each move is cut into 1 cm steps and IK is solved for every step.
- **Execute:** the joint angles are sent to the robot's motors and the physics runs for real.
- **Check and log:** did it grab the cube, lift it, carry it, and does the cube end up inside the tray? A while loop repeats this for 25 episodes and saves one log line per episode.

2.5 Keeping the cube away from the tray walls

Most early failures were the cube hitting the tray rim, so I added five simple rules:

- **Smart grasp angle:** a cube looks the same every 90°, so there are four ways to grab it. I pick the one that needs the least wrist twist, and try the others if planning fails.
- **Fly high:** while carrying, the bottom of the cube stays at least 5 cm above the tray rim.
- **Straight down:** the arm moves over the tray first and then goes straight down, never diagonally across a wall.
- **Line it up:** the cube is turned so its sides are parallel to the tray walls before it is lowered.
- **Check distances:** the planner measures the gap between gripper and table/tray on the planned path, and during the run every contact is watched.

2.6 The four IK methods in plain words

All my own solvers repeat the same loop: work out where the gripper is now, measure how far it is from the target (position and angle), and if the error is more than 1 mm or 0.6°, nudge the joints and try again. The **Jacobian J** is a table that says how much the gripper moves when each joint moves a little. The methods only differ in how they choose the nudge Δq:

Method	How it picks the next joint step	Good at	Weak at
Pseudo-inverse	$\Delta q = J^{+} \cdot K \cdot \text{error}$ ("undo" the Jacobian directly)	Simple and fast	Jumps wildly near awkward poses; ignores joint limits
DLS	$\Delta q = (J^{T}J + \lambda I)^{-1} J^{T} \cdot K \cdot \text{error}$, plus a gentle pull of each joint toward the middle of its range ($\lambda = 0.01$)	Stays calm near awkward poses; cheap	Joint limits are only encouraged, not guaranteed; λ needs tuning
QP	Same goal as DLS, solved as a small optimisation with hard rules: joint limits and joint speed limits can never be broken	Always respects the limits	Needs a solver; takes more small steps
Mink (library)	I only describe the goal ("put the TCP here") and the rules (limits, no collision with table/tray); Mink builds and solves the optimisation	Quick to use; avoids collisions too	Less control over what happens inside

One thing I learned: the null-space formula in the notes, $I - (J^{T}J + \lambda I)^{-1}J^{T}J$, is not exact when $\lambda > 0$. With it my DLS solver got stuck 2 mm from the target. Using the exact version, $I - J^{+}J$, fixed it.

3. Results

I ran three experiments, each with 25 episodes per method on the same seeds:

- **Nominal** – cube in the designed area, all five tray rules on. This is the main result.
- **Stress** – cube anywhere on the table.
- **Naive** – rules switched off on purpose (raw cube angle, travel only 6 cm high) to create failures to study.

3.1 Summary metrics

Metric	Mink	Pseudo-inverse	DLS	QP
Success – nominal	25/25	25/25	25/25	25/25
Success – stress (whole table)	25/25	25/25	25/25	25/25
Success – naive (rules off)	4/25	6/25	7/25	4/25
Average planning time per episode	56.5 ms	33.9 ms	48.4 ms	73.0 ms
IK iterations per step (average / max)	1.0 / 1	2.0 / 2	2.14 / 3	5.89 / 7
Closest gripper came to table/tray	11.3 mm	11.6 mm	11.5 mm	11.9 mm
Closest cube came to tray wall	54.3 mm	54.4 mm	53.9 mm	54.2 mm
Cube touched the tray (nominal / stress)	0 / 0	0 / 0	0 / 0	0 / 0
Joint-limit violations, naive (steps)	0	413	375	0
Speed-limit violations / top speed vs limit, naive	4 / 3.27×	0 / 0.81×	0 / 0.81×	1 / 1.44×

In plain words: **every method succeeded in all 50 normal and stress episodes**, the cube never touched the tray, and it always passed more than 5 cm from the walls. With 25 out of 25, the 95 % confidence range of the success rate is 87–100 %.

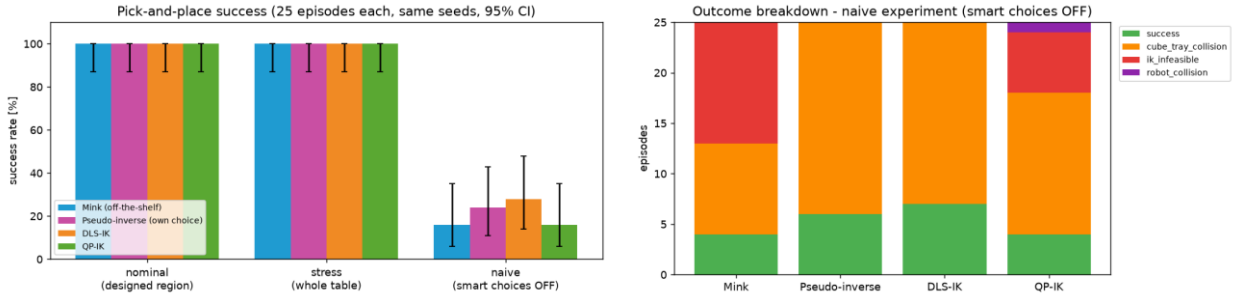


Fig. 3 – Left: success rate in each experiment (error bars = 95 % confidence). Right: what happened in each naive episode.

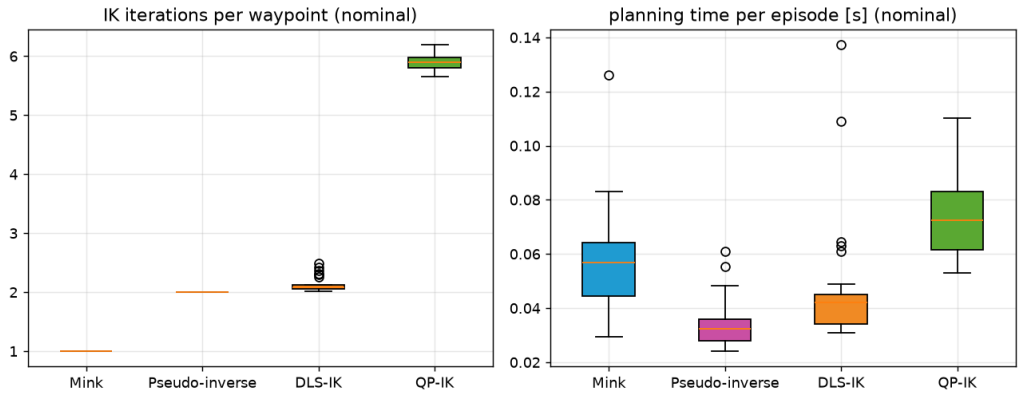


Fig. 4 – IK iterations per step and planning time per episode (nominal). Mink needs about 1 iteration, QP about 6 because its speed limit keeps every step small.

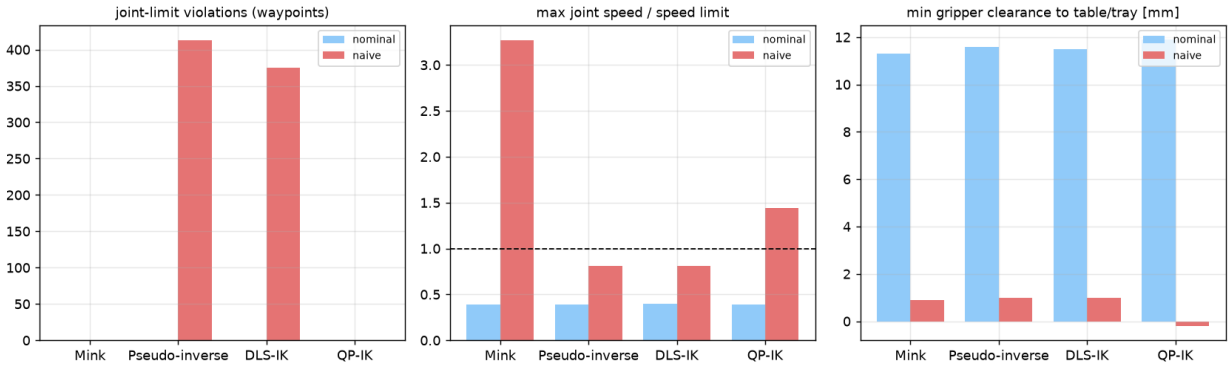


Fig. 5 – Joint-limit violations, top joint speed compared with the limit (dashed line = limit), and closest gripper distance.

3.2 What is in the logs

Every episode writes one line to *results/logs_<method>.csv* with: starting cube position and angle, success, failure reason, planning time, IK time, IK iterations (total, average, max), number of steps, joint-limit and speed-limit violations, closest gripper and cube distances, cube-tray contact, chosen grasp and place angles, and where the cube finally ended up. Two examples:

Run	Cube x, y (m)	Cube angle	Success	Failure reason	Plan time	IK iters	Gripper gap	Cube–wall gap
Mink, ep 0	0.512, 0.006	−165°	Yes	–	33 ms	1.0	11.3 mm	55.0 mm
QP naive, ep 3	0.562, 0.069	−146°	No	finger hit tray wall	98 ms	7.9	−0.2 mm	−4.9 mm

4. Failure Analysis and Fixes

To understand failures I switched my rules off (the naive test). A screenshot is saved automatically for every failed episode – 76 in total in results/failures/. Two examples:

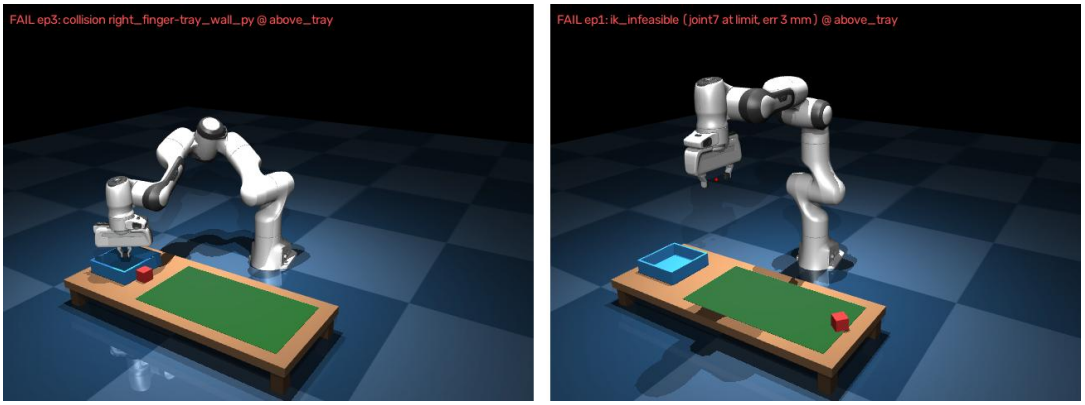


Fig. 6 – Left: QP, naive episode 3 – the cube was knocked off by the tray rim and a finger hit the wall. Right: Mink, naive episode 1 – no IK answer, because the wrist (joint 7) would need to turn past its limit.

What went wrong	Seen with	Why it happened	How I fixed / would fix it
Cube hit the tray rim	All methods	Carried only 6 cm high, so the cube scraped the wall and was knocked out	Fly 5 cm above the rim, then go straight down (0 contacts after the fix)
No IK answer (wrist at its limit)	Mink, QP	Gripping the cube at its raw angle needs more wrist twist than allowed	Use one of the 4 equivalent grasp angles and retry others
Joints pushed past their limits	Pseudo-inverse, DLS	These methods have no hard limits, so the motors clamp and the real motion differs from the plan	Pull joints to mid-range, or use QP's hard limits
Path blocked by collision check	Mink	Mink refused to move the hand through the tray rim	Add a higher in-between point or a different approach direction
Sudden fast jump (3.3× speed limit)	Mink (1 episode)	IK jumped to a different arm shape	Smaller steps, stronger pull to home posture, speed check before running
Gripper only 2 mm above table	All (early version)	Motors sagged under gravity	Gravity compensation

5. Discussion and Conclusions

Good task design mattered more than the IK method. Once the table was sized from the workspace maps and the five tray rules were in place, all four methods reached 100 % success. The real difference between them is *how they fail*:

- **Pseudo-inverse and DLS** never say no. They quietly plan through the joint limits (413 and 375 bad steps). That is why their naive success (24–28 %) looks better than QP and Mink (16 %) – but on a real robot it would be unsafe.
- **QP and Mink** respect the limits and honestly report "no solution" instead. That is the safer behaviour.
- **Which one would I use?** DLS for quick, simple jobs; QP when limits must never be broken (it costs about 1.5× the planning time of DLS); Mink when I want collision avoidance with little code.

What I learned along the way: the exact null-space formula matters for DLS, gravity compensation is needed for accurate tracking, and the workspace check must include link clearance, otherwise a floor-level table wrongly looks best. Next steps would be adding collision rules to my own QP, trying side approaches for cubes near the edge of reach, and running more episodes for tighter confidence ranges.

References

[1] K. Zakka, Mink – Python inverse kinematics based on MuJoCo, github.com/kevinzakka/mink
[2] MuJoCo physics engine, mujoco.org; MuJoCo Menagerie (Franka Emika Panda model), github.com/google-deepmind/mujoco_menagerie
[3] D. Das, Tutorial 4: Pick and Place in MuJoCo with IK – DLS-IK, QP-IK and quaternion error notes, ITR course material